



universität
uulm

Interactive Computer Graphics

Chapter 5 – Rasterization

Summer Term 2026

Timo Ropinski

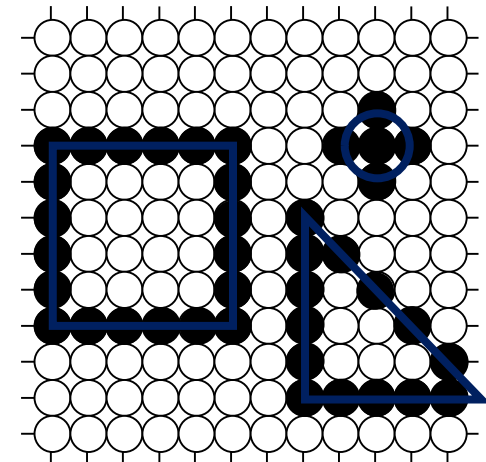


Visual Computing Group
Institute of Media Informatics

Rasterization 1/2

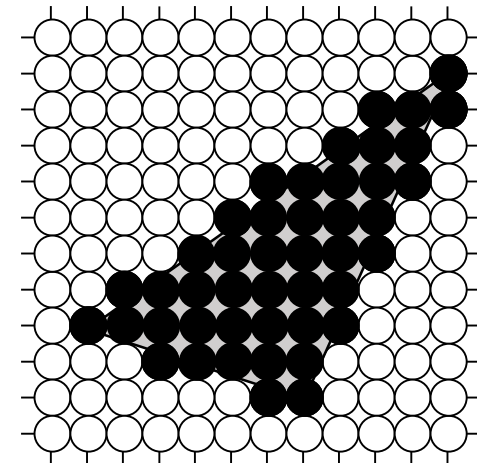
- Pixels ...
 - ... are picture elements
 - ... are the smallest elements of a raster image
 - ... do not overlap and cover the entire picture
- Advantages
 - Easy to save
 - Easy to handle (e.g. for image processing)
- Disadvantage
 - High storage space requirement
 - Transformation can lead to artifacts

**A Pixel Is *Not* A Little Square,
A Pixel Is *Not* A Little Square,
A Pixel Is *Not* A Little Square!**
[Technical Memo](#), Alvy Ray Smith (1995)



Rasterization 2/2

- Pixel-based displays require discretization of primitives into pixels
- Discretization is done by rasterization algorithms
- Rasterization is time-critical
 - Avoid expensive operations
 - Optimize for hardware implementation
 - Focus on triangles primitives (other polygons can be tessellated)
- Rasterization is quality critical
 - Goodness of approximation important



Chapter Outline

6.1 Line Rasterization

6.2 Triangle Edge Rasterization

6.3 Region Filling

6.4 Scanline-Based Triangle Rasterization

6.5 Block-Based Triangle Rasterization

References



universität
uulm

6.1 Line Rasterization

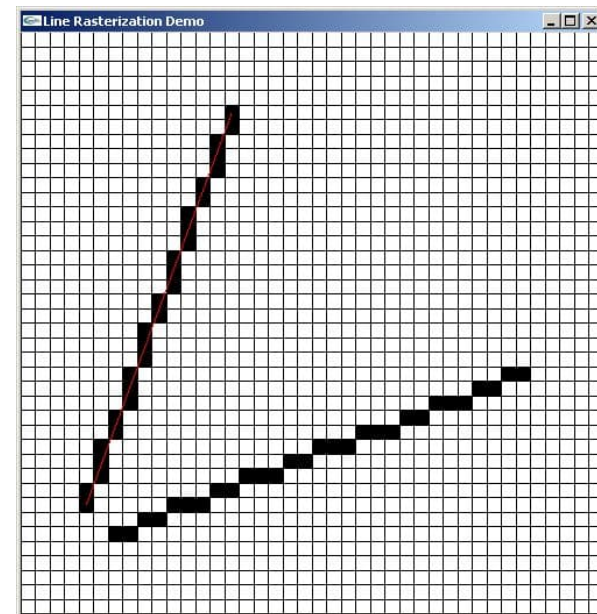
Drawing a one pixel thick line



Visual Computing Group
Institute of Media Informatics

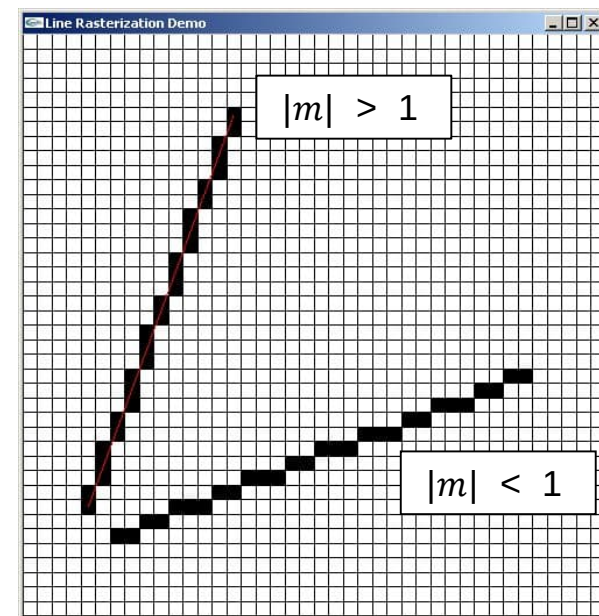
Line Rasterization

- Given two 2D endpoints in integer coordinates, determine pixels depicting 1-pixel wide line segment between endpoints
- Trivial cases
 - Horizontal line segments
 - Vertical line segments
 - Diagonal line segments
- Non-trivial cases
 - General case



Quality Criteria

- For all line segments with a slope $|m| \leq 1$, one pixel should be drawn per column
- For all line segments with a slope $|m| \geq 1$, one pixel per line should be drawn
- All line segments should have uniform width regardless of slope and length



Naïve Algorithm 1/2

- Utilization of the Cartesian normal form of the straight line equation

$$y = m \cdot x + B$$

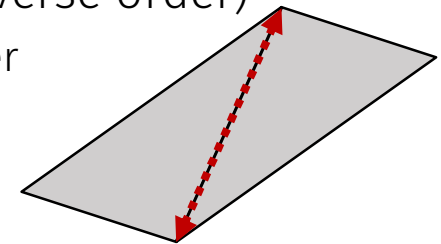
- Assumptions (without restriction of generality)
 - Line segments lie between (x_0, y_0) and (x_1, y_1)
 - Lines are drawn from left to right $x_0 < x_1$

```
void drawLine(int x0, int y0, int x1, int y1) {  
    float dx = x1 - x0;           // Calculate change in x  
    float dy = y1 - y0;           // Calculate change in y  
    float m = dy / dx;            // Calculate slope of the line  
    float B = y0 - m * x0;         // Calculate y-intercept of the line  
    for (int x = x0; x <= x1; x++) { // Loop from start x to end x  
        writePixel(x, round(m * x + B)); // Draw pixel at (x, y)  
    }  
}
```

Naïve Algorithm 2/2

■ Downsides

- Does not work for $x_0 = x_1$
- Does not work for $|m| > 1$, since gaps arise (solution: swap the roles of x and y for $|m| > 1$)
- In each iteration, a multiplication $m \cdot x$ is performed (high computational effort)
- Using floating-point calculations (additional type conversions necessary)
- Rounding prevents reversibility (reversibility: reversing endpoints draws same pixels in reverse order)
 - When drawing and then deleting a line, pixels may be left over
 - Line width might change when drawing opposing edges



Digital Difference Analyzer (DDA) 1/3

- Reuse the last computed y to avoid multiplications in the loop

$$x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$$

```
void drawLine(int x0, int y0, int x1, int y1) {  
    float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope  
    float y = y0; // Start y at y0  
    for (int x = x0; x <= x1; x++) { // Loop from x0 to x1 and increment x  
        writePixel(x, round(y)); // Draw pixel at (x, rounded y)  
        y += m; // Increment y by slope per step in x  
    }  
}
```

Digital Difference Analyzer (DDA) 2/3

- Algorithm can be extended to relax endpoint order

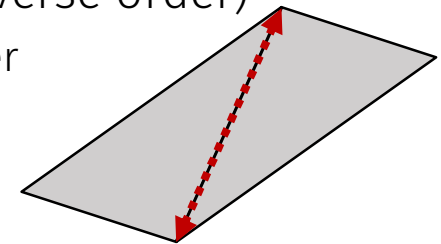
```
void drawLine(int x0, int y0, int x1, int y1) {  
    int sx = (x0 < x1 ? 1 : -1);  
    float sm = sx * float(y1 - y0) / float(x1 - x0);  
    float y = y0;  
    for (int x = x0; x != x1 + sx; x += sx) {  
        writePixel(x, round(y));  
        y += sm;  
    }  
}
```

// Determine x direction
// Calculate slope multiplied by step
// Start y at y0
// Loop from x0 to x1 with step sx
// Draw pixel at (x, rounded y)
// Increment y by slope*step per x-step

Digital Difference Analyzer (DDA) 3/3

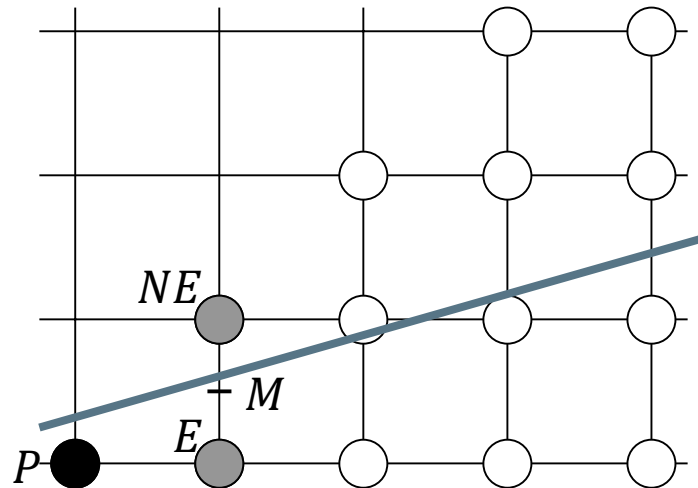
■ Downsides

- Does not work for $x_0 = x_1$
- Does not work for $|m| > 1$, since gaps arise (solution: swap the roles of x and y for $|m| > 1$)
- In each iteration, a multiplication ~~$m \cdot x$~~ is performed (high computational effort)
- Using floating-point calculations (additional type conversions necessary)
- Rounding prevents reversibility (reversibility: reversing endpoints draws same pixels in reverse order)
 - When drawing and then deleting a line, pixels may be left over
 - Line width might change when drawing opposing edges



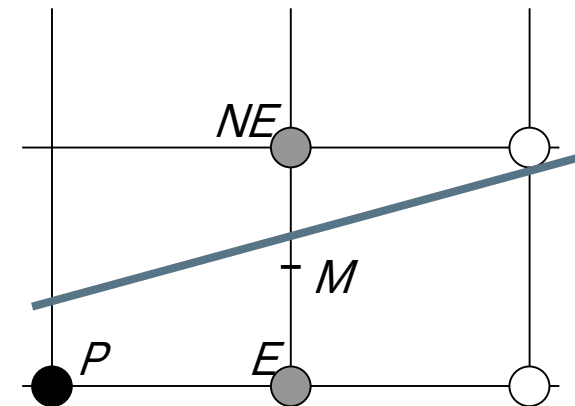
Midpoint-Line Algorithm 1/2

- Assumption 1: $0 \leq m \leq 1$
(without loss of generality, as otherwise mirroring on axes/diagonals)
- Lower left end point $P_0 = (x_0, y_0)$,
upper right end point $P_1 = (x_1, y_1)$
- $P_p = (x_p, y_p)$ is last drawn pixel
- After P_p one of the following two pixels has to be chosen
 - $E = (x_p + 1, y_p)$
 - $NE = (x_p + 1, y_p + 1)$



Midpoint-Line Algorithm 2/2

- Observation
 - E is closer to the line if M is above the line
 - NE is closer to the line if M is below the line
 - Vertical distance between pixel to choose and the actual line segment is always < 0.5
- Goal: find an efficient calculation rule to determine if M is above or below line segment



Implicit Function

- Line equation usually given as explicit function $f(x)$

$$f(x) = y = m \cdot x + B = \frac{dy}{dx} \cdot x + B, \text{ with } dx = x_1 - x_o \text{ and } dy = y_1 - y_o$$

- Line equation can also be written as implicit function $F(x, y)$

$$F(x, y) = a \cdot x + b \cdot y + c = 0$$

$$\Leftrightarrow F(x, y) = dy \cdot x - dx \cdot y + B \cdot dx = 0$$

$$\Rightarrow a = dy, \quad b = -dx, \quad c = B \cdot dx, \quad a > 0 \text{ for } y_1 > y_o$$

- $F(x, y)$ has the following properties

$$F(x, y) \begin{cases} > 0 \text{ under line} \\ = 0 \text{ on line} \\ < 0 \text{ above line} \end{cases}$$

- Pixel selection done based on value of F at M : $F(M) = F(x_p + 1, y_p + \frac{1}{2})$

Decision Variable d

- Sign of $F(M)$ is sufficient to decide which point is chosen
- Define decision variable d as $d = F(M) = F(x_p + 1, y_p + \frac{1}{2})$
 - If $d > 0$, select NE (M below the line)
 - If $d < 0$, select E (M above the line)
 - If $d = 0$, consistently choose either NE or E
- Update decision variable d
 - Depending on choice of NE or E and value of d for the selected pixel, d is calculated for the next grid line ($x_p + 2$) from the current d

Update of d for E

- Previous decision variable was given by

$$d_{old} = F\left(x_p + \mathbf{1}, y_p + \frac{1}{2}\right) = a \cdot (x_p + \mathbf{1}) + b \cdot \left(y_p + \frac{1}{2}\right) + c$$

- M is incremented by $\mathbf{1}$ in the x -direction, resulting in

$$d_{new} = F\left(x_p + \mathbf{2}, y_p + \frac{1}{2}\right) = a \cdot (x_p + \mathbf{2}) + b \cdot \left(y_p + \frac{1}{2}\right) + c$$

- Incremental difference between d_{new} and d_{old}

$$\begin{aligned} d_{new} &= d_{old} + \mathbf{a} \\ \Delta E &:= a = dy \end{aligned}$$

- Calculate decision variable when choosing E incremental

$$d_{new} = d_{old} + \Delta E$$

Update of d for NE

- Previous decision variable was given by

$$d_{old} = F\left(x_p + 1, y_p + \frac{1}{2}\right) = a \cdot (x_p + 1) + b \cdot \left(y_p + \frac{1}{2}\right) + c$$

- M is incremented by 1 in the x - and y -direction, resulting in

$$d_{new} = F\left(x_p + 2, y_p + \frac{3}{2}\right) = a \cdot (x_p + 2) + b \cdot \left(y_p + \frac{3}{2}\right) + c$$

- Incremental difference between d_{new} and d_{old}

$$\begin{aligned} d_{new} &= d_{old} + a + b \\ \Delta NE &:= a + b = dy - dx \end{aligned}$$

- Calculate decision variable when choosing NE incremental

$$d_{new} = d_{old} + \Delta NE$$

Proceeding

- In each step, the algorithm selects between two pixels NE and E based on sign of decision variable d
- Depending on the choice, decision variable is updated by adding difference ΔNE or ΔE

Initializing d

- First midpoint M is located at $(x_0 + 1, y_0 + \frac{1}{2})$

$$\begin{aligned} F\left(x_0 + 1, y_0 + \frac{1}{2}\right) &= a \cdot (x_0 + 1) + b \cdot \left(y_0 + \frac{1}{2}\right) + c = a \cdot x_0 + b \cdot y_0 + c + a + \frac{b}{2} \\ &= F(x_0, y_0) + a + \frac{b}{2} \end{aligned}$$

- As (x_0, y_0) lies on line $F(x_0, y_0) = 0$ holds, and

$$d_{start} = a + \frac{b}{2} = dy - \frac{dx}{2}$$

- To eliminate fraction in d_{start} it is multiplied by 2

$$d_{start} = 2 \cdot a + b = 2 \cdot dy - dx$$

- Thus, decision variable d , all constants and the increments ΔE and ΔNE must also be multiplied by 2, which does not affect sign of d

$$F(x, y) = 2 \cdot (a \cdot x + b \cdot y + c) = 0$$

Using d

- Midpoint line algorithms uses d as follows

$$d_{start} = 2 \cdot dy - dx$$

$$\Delta NE = 2 \cdot (dy - dx)$$

$$\Delta E = 2 \cdot dy$$

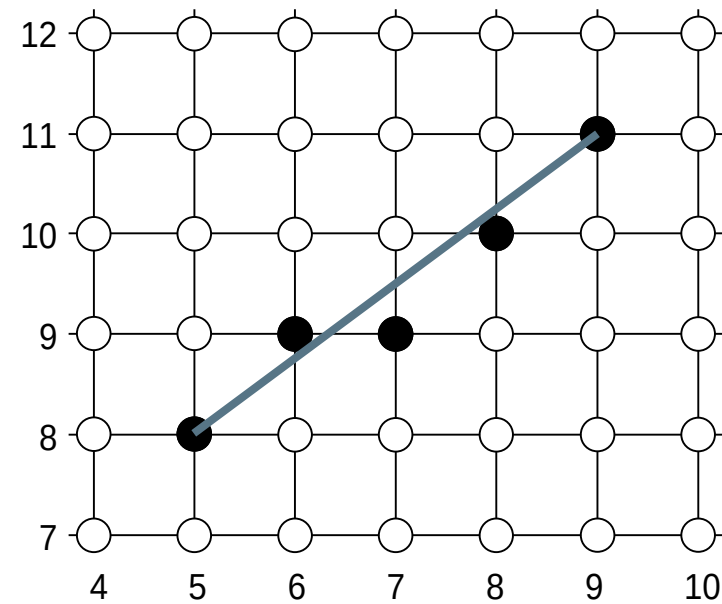
- When E is chosen ($d_{old} \leq 0$): $d_{new} = d_{old} + 2 \cdot dy$
- When NE is chosen ($d_{old} > 0$): $d_{new} = d_{old} + 2 \cdot (dy - dx)$

Pseudo Code – Simple Version

```
void midPointLine(int x0, int y0, int x1, int y1) {
    int dx = x1 - x0;           // Calculate delta x
    int dy = y1 - y0;           // Calculate delta y
    int d = 2 * dy - dx;        // Initial decision variable
    int dE = 2 * dy;             // Increment used for move to E
    int dNE = 2 * (dy - dx);    // Increment used for move to NE
    int x = x0, y = y0;         // Start position
    writePixel(x, y);           // Plot the first pixel
    while (x < x1) {             // Loop through from x0 to x1
        if (d <= 0) {            // Decision is East
            d += dE; x++;
        } else {                 // Decision is North East
            d += dNE; x++; y++;
        }
        writePixel(x, y);        // Plot each computed pixel
    }
}
```

Example

- Draw line from (5, 8) to (9, 11)



Evaluation

- Necessary arithmetic operations
 - As only integer values are used, only integer operations are necessary
 - Additions/subtractions only and multiplication by 2 (left shift by 1 bit position)
 - No multiplications in the loop
- Reversibility
 - Order of endpoints can be made reversible
 - Line from P_0 to P_1 has the same pixels as line from P_1 to P_0

⇒ when drawing from right to left select SW pixels for $d = 0$

Pseudo Code - Reversible

```
void drawLine(int x0, int y0, int x1, int y1) {  
    int dx = abs(x1 - x0);  
    int sx = (x0 < x1 ? 1 : -1);  
    int dy = abs(y1 - y0);  
    int sy = (y0 < y1 ? 1 : -1);  
    int d = 2 * dy - dx;  
    int dE = 2 * dy;  
    int dNE = 2 * (dy - dx);  
    int x = x0, y = y0;  
    writePixel(x, y);  
    while (x != x1) {  
        if ((d < 0) || ((d == 0) && (sx == 1))) {  
            d += dE; x += sx;  
        } else {  
            d += dNE; x += sx; y += sy;  
        }  
        writePixel(x, y);  
    }  
}
```

// Calculate absolute delta x
// Set step x based on direction
// Calculate absolute delta y
// Set step y based on direction
// Initial decision variable
// Increment for move East or West
// Increment for move NE or SW
// Start positions
// Plot the first pixel
// Loop until target x reached
// Decide East or West

// Decide North-East or South-West

// Plot each computed pixel

6.2 Triangle Edge Rasterization

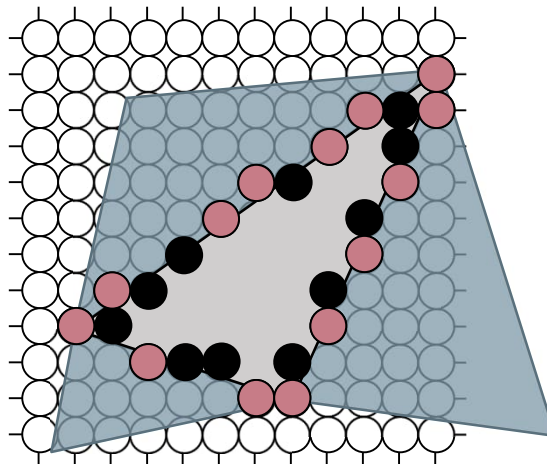
Drawing a one pixel thick triangle edge



Visual Computing Group
Institute of Media Informatics

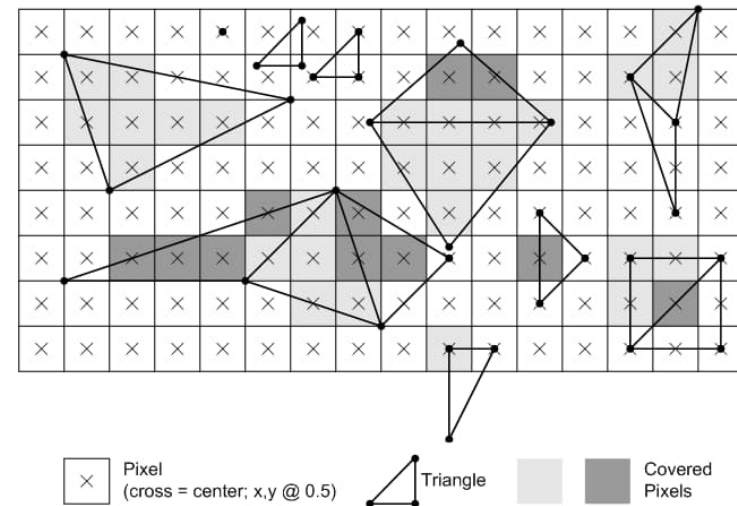
Triangle Edges 1/2

- Using midpoint line algorithm for triangle edge rasterization
 - Provides pixels closest to triangle edge
 - Does not consider if pixel lies inside or outside triangle
- ⇒ Edge pixels can penetrate into interior of adjacent triangles



Triangle Edges 2/2

- Midpoint line algorithm must consider inside vs. outside pixels
 - Adaptation of the midpoint line algorithm
 - Restriction to pixels strictly inside triangle
 - Must be applied to line rasterization also for consistency reasons
- Definition of triangle inside:
edge pixel belongs to inside if:
edge half-plane lies below / right of edge
 - see [Top-Left Rule](#)
 - “top || left”, “top & left”, “left & left”



Edge Rasterization 1/3

- Triangle edge rasterization can exploit edge coherence
 - Two edges of triangle intersect a scanline
 - Same edges intersected by scanline i likely also intersect scanline $i + 1$
- Due to coherence, intersections can be calculated incrementally
 - For each increase of $y \rightarrow y + 1$
 x can be computed through edge slope A : $x = x + 1/A$

```
void leftEdgeScan(int x0, int y0, int x1, int y1) { // y0 < y1
    float x = x0; // Start x at x0
    float dx dy = (x1 - x0) / (y1 - y0); // Calculate dx/dy = 1/A
    for (int y = y0; y < y1; y++) { // Loop from y0 to y1
        writePixel(ceil(x), y); // Plot pixel at (ceil(x), y)
        x += dx dy; // Increment x by dx/dy
    }
}
```

Edge Rasterization 2/3

- Integer arithmetic can be used for (left) edges with slopes > 1
 - Consider $dx dy = (x_{max} - x_{min}) / (y_{max} - y_{min})$
 - x is only changed by $dx dy$
 $\Rightarrow x$ can be represented as a fraction with denominator $(y_{max} - y_{min})$
 - Let a be the integer part
 $d := y_{max} - y_{min}$ the denominator, and
 $n := x_{max} - x_{min}$ the numerator of $dx dy$
 $\Rightarrow x = (a \cdot d + b) / d = a \cdot d / d + b / d = a + b / d$ with $b < d$
 - When incrementing x by $dx dy$, counter b is always incremented by n
 - If $b / d > 1$ (i.e., $b > d$)
 - a is incremented by 1
 - counter b is decreased by d

Edge Rasterization 3/3

```
void leftEdgeScan(int x0, int y0, int x1, int y1) {    // slope > 1
    int d = y1 - y0;                                // denominator (Nenner)
    int n = x1 - x0;                                // numerator (Zähler)
    int a = x0;                                     // integer part of x
    int b = d;                                       // fraction part of x plus d
    for (int y = y0; y < y1; y++) {                // Loop from y0 to y1
        writePixel(a, y);                          // Plot pixel at (a, y)
        b += n;                                     // Increment b by numerator to handle x = x + 1/A
        if (b >= d) { // Check if b exceeds or equals denominator
            a++;                                     // Increment integer part of x
            b -= d; // Adjust fraction part back by denominator
        }
    }
}
```



universität
uulm

6.3 Region Filling

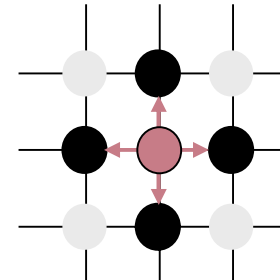
Homogeneous coloring of regions



Visual Computing Group
Institute of Media Informatics

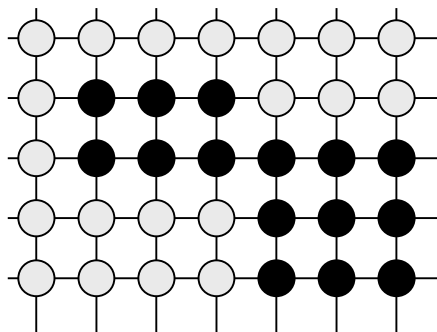
Region Types 1/2

- 4-connected
 - All pixels of region can be reached by following movements
 - Left, right, bottom, top

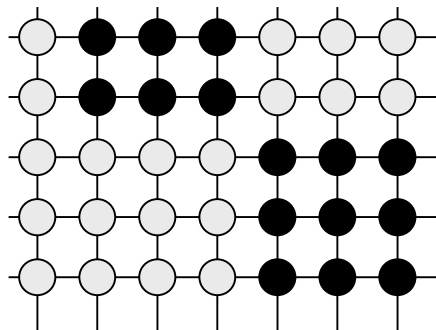


Region Types 2/2

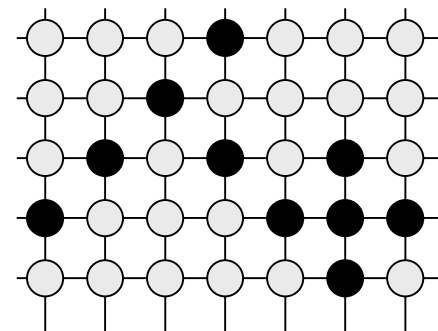
- Examples



4-connected



8-connected

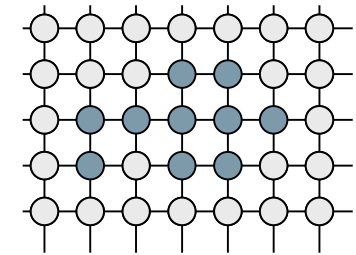


8-connected

Region Spezification

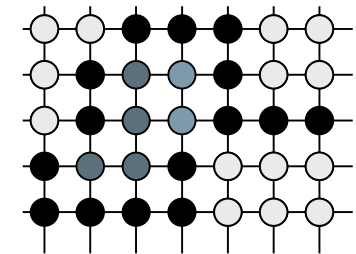
■ Color-based

- Starting at pixel P , determine largest contiguous region which pixels have a specific color
- Color assignment done by flood fill algorithms



■ Border-based

- Starting at pixel P , determine the largest contiguous region which pixels have a color different from the border color
- Color assignment done by boundary fill algorithms



Flood Fill Algorithm

```
void floodFill4(int x, int y, Color oldC, Color newC) {  
    if ((x < 0) || (x >= width)) return; // Return if x is out of bounds  
    if ((y < 0) || (y >= height)) return; // Return if y is out of bounds  
  
    if (oldC == readPixel(x, y)) { // Check if the current color matches oldC  
        writePixel(x, y, newC); // Change the color of the pixel  
        floodFill4(x, y-1, oldC, newC); // Recursively fill up  
        floodFill4(x, y+1, oldC, newC); // Recursively fill down  
        floodFill4(x+1, y, oldC, newC); // Recursively fill right  
        floodFill4(x-1, y, oldC, newC); // Recursively fill left  
    }  
}
```

Boundary Fill Algorithm

```
void boundaryFill4(int x, int y, Color oldC, Color newC) {  
    if ((x < 0) || (x >= width)) return; // Return if x is out of bounds  
    if ((y < 0) || (y >= height)) return; // Return if y is out of bounds  
  
    Color c = readPixel(x, y);  
    if (c == oldC) return; // Return if pixel is a boundary pixel  
    if (c == newC) return; // Return if pixel has been processed  
  
    writePixel(x, y, newC); // Change the color of the pixel  
    boundaryFill4(x, y-1, oldC, newC); // Recursively fill up  
    boundaryFill4(x, y+1, oldC, newC); // Recursively fill down  
    boundaryFill4(x+1, y, oldC, newC); // Recursively fill right  
    boundaryFill4(x-1, y, oldC, newC); // Recursively fill left  
}
```

Fill Algorithms

- High number of recursions can lead to stack overflow
- Use smarter data structures, i.e., queues and stacks



universität
uulm

6.4 Scanline-Based Triangle Rasterization

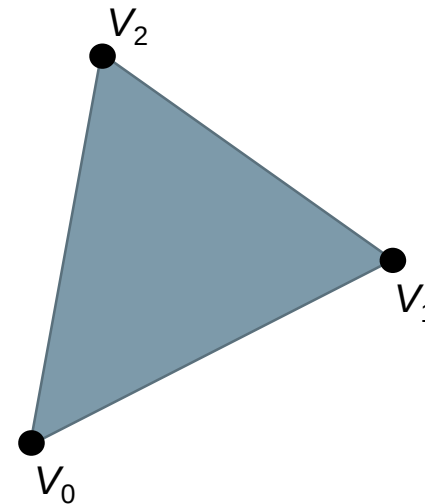
Segment-based rasterization of the simplest type of polygon



Visual Computing Group
Institute of Media Informatics

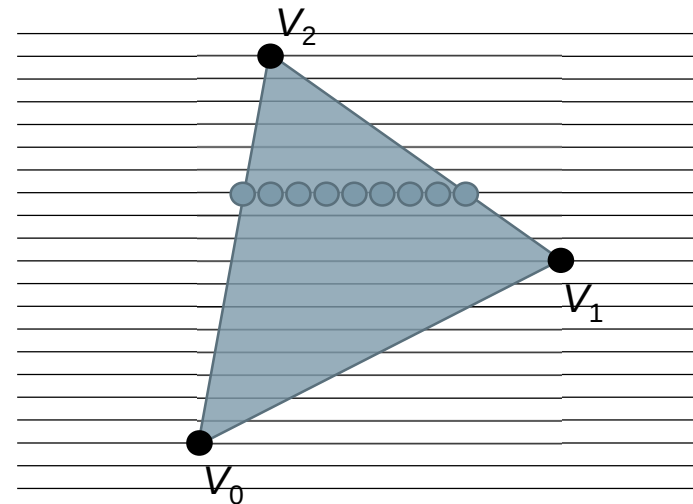
Triangle Rasterization

- Triangle rasterization
 - Rasterize filled triangles is main task in real-time rendering
 - Realization through graphics hardware desired
 - General polygons split into triangles through tessellation
- Rasterization requires three steps
 - Determine triangle edges
 - Fill triangle's interior
 - Interpolate information over triangle (e.g., colors, z-values)



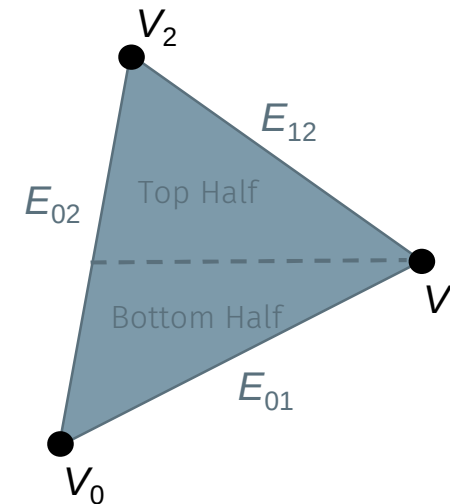
Scanline-Based Rasterization

- Rasterization through scanline conversion exploits spatial coherence
 - Pixels of a scanline are colored similarly
 - Adjacent scanlines span similar pixels
 - Adjacent scanlines are colored similarly
- Scanline segments can be calculated through modified midpoint line algorithm
 - Rasterize triangle edges to obtain edge pixels
 - Generate pixels of scanlines between each pair of edge pixels



Algorithm Overview

- Observation: triangle can be split into top and bottom half
 - For each half two edge pixels and one segment exist
 - Segments differ only slightly, depending on edge slope
- Procedure
 - Sort triangle vertices based on y -coordinate
 $\Rightarrow ((x_0, y_0), (x_1, y_1), (x_2, y_2))$ with $y_0 \leq y_1 \leq y_2$
 - Rasterize lower half between E_{01} and E_{02}
 - Rasterize upper half between E_{02} and E_{12}



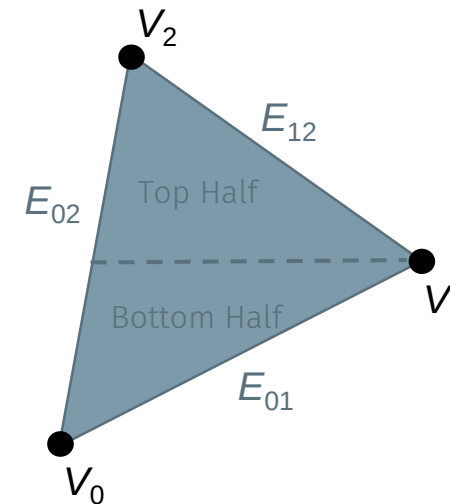
Slope Calculation

- Calculate slope for each triangle edge: $1/A = dx/dy$

- $\Delta_{02} = (x_2 - x_0)/(y_2 - y_0)$
- $\Delta_{01} = (x_1 - x_0)/(y_1 - y_0)$
- $\Delta_{12} = (x_2 - x_1)/(y_2 - y_1)$

- Calculate number of scanlines for each triangle half

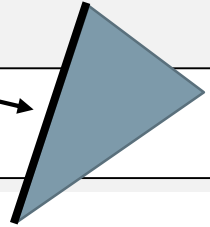
- $N_{top} = y_2 - y_1$
- $N_{bottom} = y_1 - y_0$



Pseudo Code

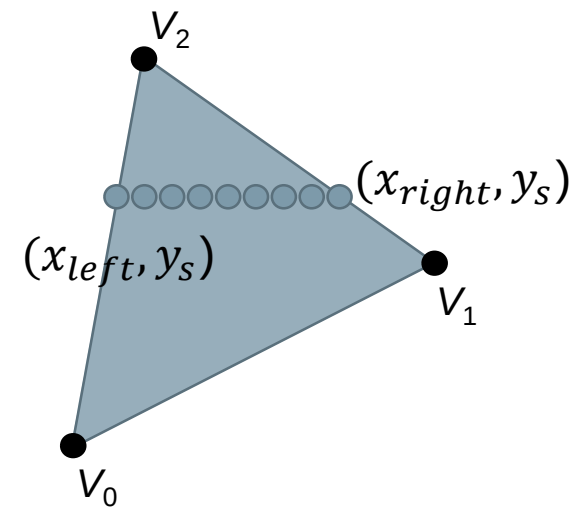
```
void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) {
    float XLeft = x0;           // Start position for left x
    float XRight = x0;          // Start position for right x
    int Y = y0;                 // Start position for y
    float DeltaLeft = delta02;   // Delta for left x for bottom half
    float DeltaRight = delta01;  // Delta for right x for bottom half
    for (int i = 0; i < Nbottom; i++) { // Loop for bottom half of the triangle
        processSegment(Y, round(XLeft), round(XRight)); // Process current segment
        XLeft += DeltaLeft; // Increment left x by delta left
        XRight += DeltaRight; // Increment right x by delta right
        Y += 1; // Increment y
    }
    DeltaRight = delta12; // Delta for right x for top half
    for (int i = 0; i <= Ntop; i++) { // Loop for top half of the triangle
        processSegment(Y, round(XLeft), round(XRight)); // Process current segment
        XLeft += DeltaLeft; // Increment left x by delta left
        XRight += DeltaRight; // Increment right x by delta right
        Y += 1; // Increment y
    }
}
```

Assumption: main axis is left
Otherwise: switch left and right



Segment Rasterization

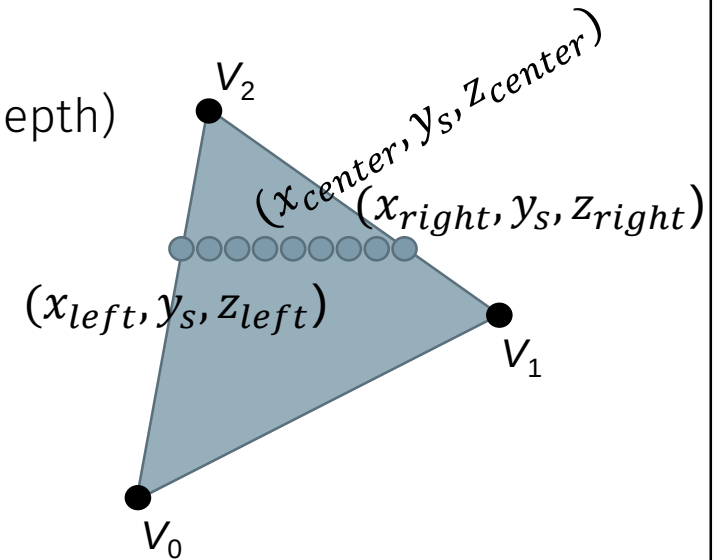
- Each segment generates fragments
- Rendering pipeline processes generated fragments (e.g., depth testing, alpha blending)



```
void processSegment(int y_s, int x_left, int x_right) {  
    for (int x = x_left; x < x_right; x++) {           // Loop from x_left to x_right  
        processFragment(x, y_s);                       // Process each fragment at (x, y_s)  
    }  
}
```

Vertex Attribute Interpolation

- During rasterization vertex attributes (e.g., color, depth) must be interpolated over triangle
- Bilinear interpolation can be used
 1. Interpolate attributes along both edges
 - $z_{left} = z_0 + \frac{(y_s - y_0)}{(y_2 - y_0)} \cdot (z_2 - z_0)$
 - $z_{right} = z_1 + \frac{(y_s - y_1)}{(y_2 - y_1)} \cdot (z_2 - z_1)$
 2. Interpolate attributes along segment
 - $z_{center} = z_{left} + \frac{(x_{center} - x_{left})}{(x_{right} - x_{left})} \cdot (z_{right} - z_{left})$



Incremental Interpolation 1/3

- Spatial coherence allows for an incremental attribute interpolation
- Example: consider triangle's plane equation for calculating z increments
 - $Ax + By + Cz + D = 0$
 - With plane coefficients A, B, C : $[A, B, C] = (V_2 - V_0) \times (V_1 - V_0)$
 - Resolve for z : $z = (-Ax - By - D)/C$
 - Calculate $dzdx$ as change of z in case of horizontal change by $+1$
 - Consider $z_i \rightarrow z_{i+1}$ at $x_i \rightarrow x_{i+1}$ where $x_{i+1} = x_i + 1$:
 $((-Ax_i - By - D)/C) - ((-Ax_{i+1} - By - D)/C) = -A/C =: dzdx$
 - Calculate $dzdy$ as change of z in case of vertical change by $+1$
 - Consider $z_i \rightarrow z_{i+1}$ at $y_i \rightarrow y_{i+1}$ where $y_{i+1} = y_i + 1$:
 $((-Ax - By_i - D)/C) - ((-Ax - By_{i+1} - D)/C) = -B/C =: dzdy$

Incremental Interpolation 2/3

- Increments $dzdx$ and $dzdy$ are used to incrementally modify given z value, i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$
- Case 1: next fragment in scanline (horizontal change by +1)
 - $x_i \rightarrow x_{i+1} \Rightarrow z := z_i + dzdx$
- Case 2: first fragment on next scanline
 - $x_i \rightarrow x_0 \wedge y_i \rightarrow y_{i+1} \Rightarrow z := z_i + dzdy - (x_i - x_0) \cdot dzdx$
- $dzdx$ and $dzdy$ can be precomputed for both triangle halves

Incremental Interpolation 3/3

- All vertex attributes can be interpolated incrementally analogous to depth
 - Colors (r, g, b, a)
 - Texture coordinates (u, v)
 - ...
- For each attribute θ define
 - $d\theta dx$
 - $d\theta dy$
 - With $\theta \in \{z, r, g, b, a, n_x, n_y, n_z, u, v, \dots\}$
- $d\theta dx$ and $d\theta dy$ can be precomputed for both triangle halves



universität
uulm

6.5 Tile-Based Triangle Rasterization

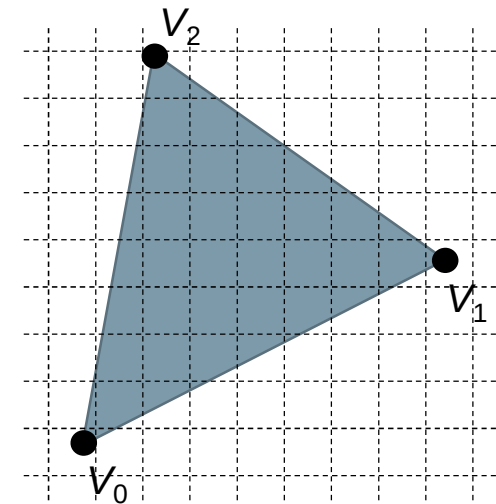
Parallel tile-based triangle rasterization



Visual Computing Group
Institute of Media Informatics

Tile-Based Triangle Rasterization

- Modern GPUs have a block-wise structure, where one processor is responsible for one screen tile of adjacent pixels
- Segment-based triangled rasterization is difficult to parallelize
- Tile-based rasterization is a better fit with modern GPUs
- Advantage
 - Different tiles can be rasterized independently
- Drawback
 - Vertex attribute interpolation can only reuse information from within the same tile



Edge-Based Triangle Representation 1/2

- Edge-based area representation is possible through implicit edge functions
- Implicit edge function can be defined based on two edge vertices

$$V_0 = (x_0, y_0) \text{ and } V_1 = (x_1, y_1)$$

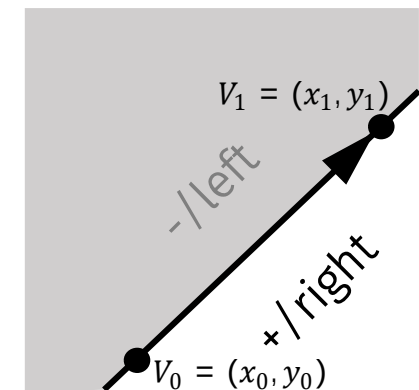
- For a given coordinate (x, y) , the implicit edge function is defined as

$$E(x, y) = (y_1 - y_0) \cdot x - (x_1 - x_0) \cdot y + B_{01} \cdot (x_1 - x_0)$$

with $B_{01} = y_0 - m_{01} \cdot x_0$ and $m_{01} = (y_1 - y_0)/(x_1 - x_0)$

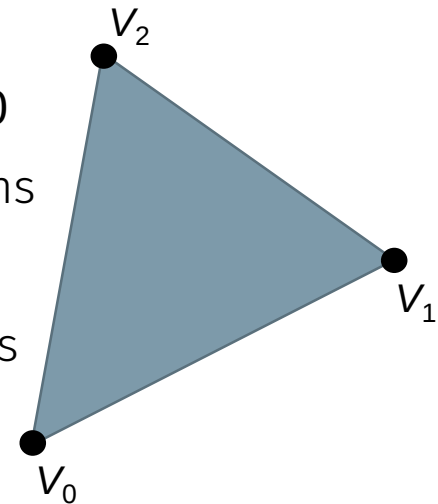
- $E(x, y)$ has the following properties

$$E(x, y) \begin{cases} > 0: (x, y) \text{ is right of edge} \\ = 0: (x, y) \text{ is on the edge} \\ < 0: (x, y) \text{ is left of edge} \end{cases}$$



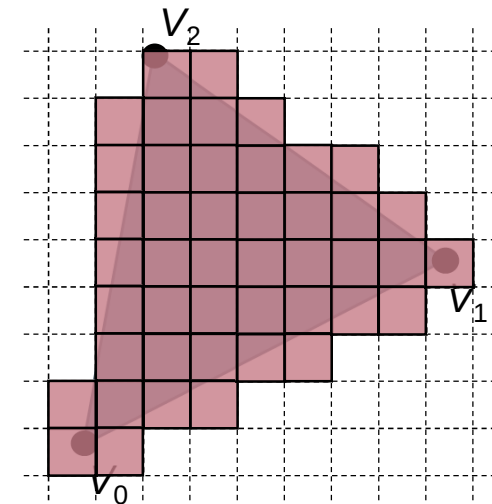
Edge-Based Triangle Representation 2/2

- To represent a triangle, three implicit edge functions are combined
 - $E_{01}(x, y) = (y_1 - y_0) \cdot x - (x_1 - x_0) \cdot y + B_{01} \cdot (x_1 - x_0)$
 - $E_{12}(x, y) = (y_2 - y_1) \cdot x - (x_2 - x_1) \cdot y + B_{12} \cdot (x_2 - x_1)$
 - $E_{20}(x, y) = (y_0 - y_2) \cdot x - (x_0 - x_2) \cdot y + B_{20} \cdot (x_0 - x_2)$
- A given pixel at location (x, y) is inside the triangle if
$$E_{01}(x, y) < 0 \wedge E_{12}(x, y) < 0 \wedge E_{20}(x, y) < 0$$
- Formulation can be evaluated in parallel at many positions
- Formulation can be extended to arbitrary convex polygons by using as many implicit edge functions as edges exist



Rasterization Procedure

1. Determine tiles which are (partially) covering triangle (e.g., easiest approximation would be bounding box)
2. For each covering tile
 - For each pixel
 - If all implicit edge functions evaluate to < 0
 - Interpolate vertex attributes
 - Draw pixel



Chapter Summary

- Pixel-based displays require the rasterization of primitives
 - Rasterization fidelity is critical for image quality
 - Rasterization is time-critical and should be optimized for hardware
- Focus on triangles due to speed and ease of tessellation of other polygons
- Segment-based triangle rasterization avoids overlap between adjacent triangles
 - Midpoint scheme can be used to rasterize triangle edges
- Block-based triangle rasterization allows for parallel processing on graphics hardware



universität
uulm

References

Literature and other sources used in this chapter



Visual Computing Group
Institute of Media Informatics

References

- Text Books

- Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and Practice, Addison-Wesley.

- Research Publications

- Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter. In Seminal graphics: pioneering efforts that shaped the field (pp. 1-6).
- Pineda, J. (1988, June). A parallel algorithm for polygon rasterization. In Proceedings of the 15th annual conference on Computer graphics and interactive techniques (pp. 17-20).



Slide for Future: Sustainability in CG (@ [EG25](#))

[Submitting](#)[Program](#)[Attending](#)[Organization](#)[Sponsorship](#)

Sustainability Panel

Join us at Eurographics 2025 for this new fishbowl-style discussion on sustainability in computer graphics, featuring leading voices from industry and research. Thereby, sustainability focuses on environmental aspects, i.e., addressing resource consumption and discussing how graphics technology can complement fields like environmental and climate research. In this open dialogue format, experts will share insights on (reducing) energy footprints and explore how our core research questions may evolve for a more environmentally cautious future. Together, we'll develop practical guidelines to inspire more sustainable practices in our community. Be part of the conversation that shapes a greener future in computer graphics — your ideas could spark the next breakthrough!

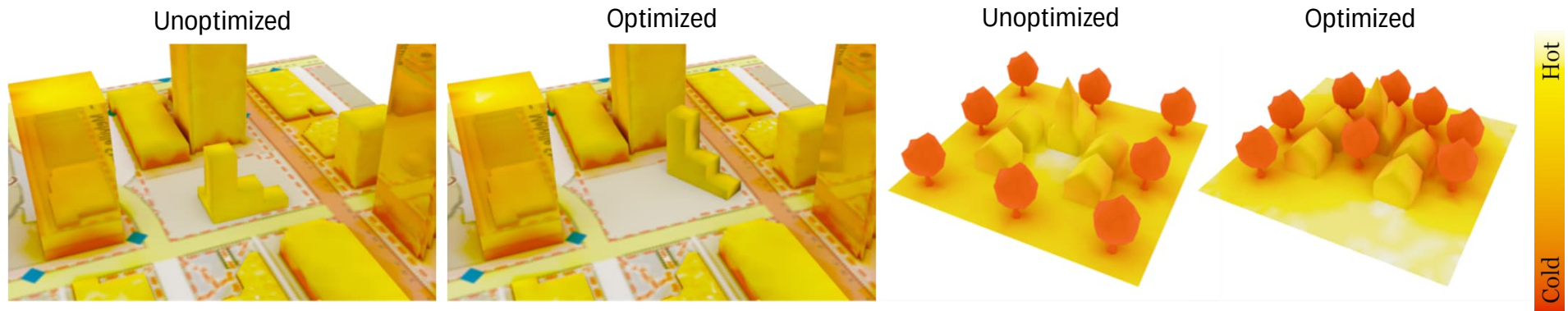
Our invited panelists:

- **Ursula Augsdörfer**
Associate Professor at TU Graz (Institute of Visual Computing), head of the Intelligent Geometry and Simulation group and elected member of the EG Executive Committee
- **Tamy Boubekeur**
Senior Director and Senior Principal Research Scientist at Adobe Research and Professor at École Polytechnique
- **Jonathan Harris**
Founder and CEO of dupeVFX, actively tracking CO2 emissions of their productions and the first VFX studio with a "B Corporation" certification
- **Daniel Schien**
Associate Professor at the University of Bristol (Sustainable Information and Communication Technologies) and technical development lead at DIMPACT.org
- **You**
Welcome to the discussion! Feel free to bring in your unique view and share your ideas.

also @ [IEEE VR](#)



Slide for Future: Sustainability in CG (@ EG25)

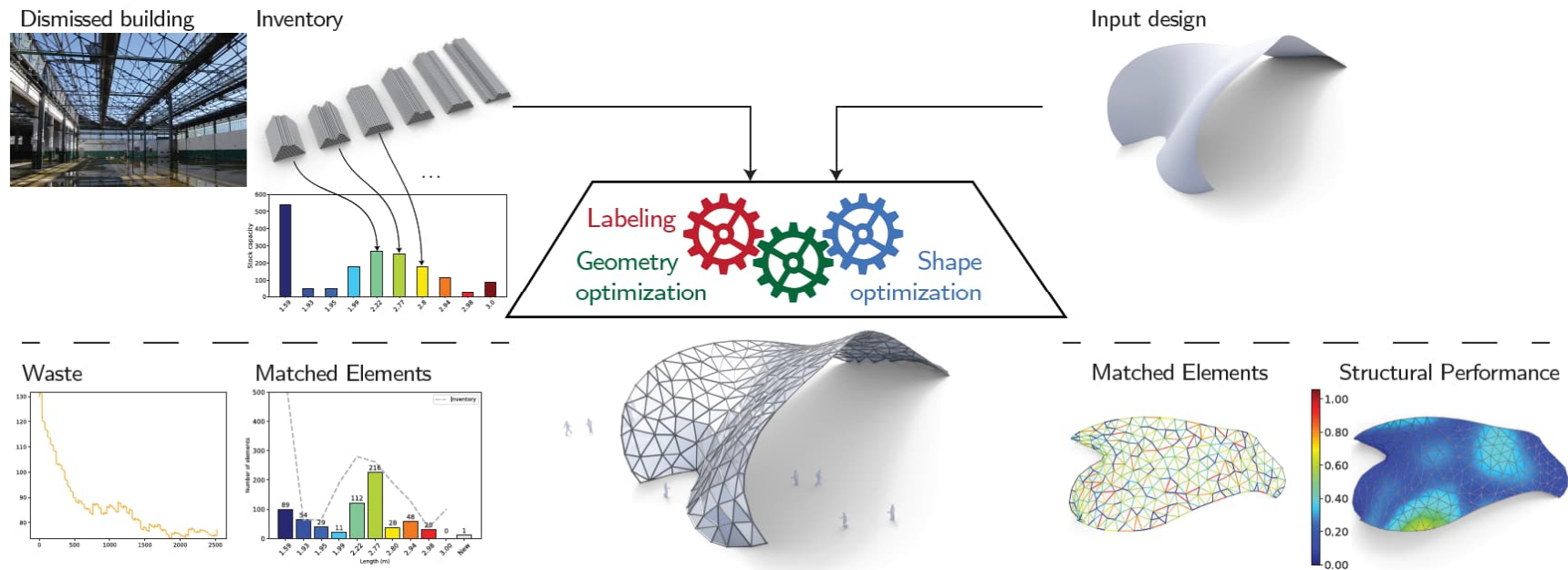


[Inverse Simulation of Radiative Thermal Transport](#)

Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn



Slide for Future: Sustainability in CG (@ EG25)

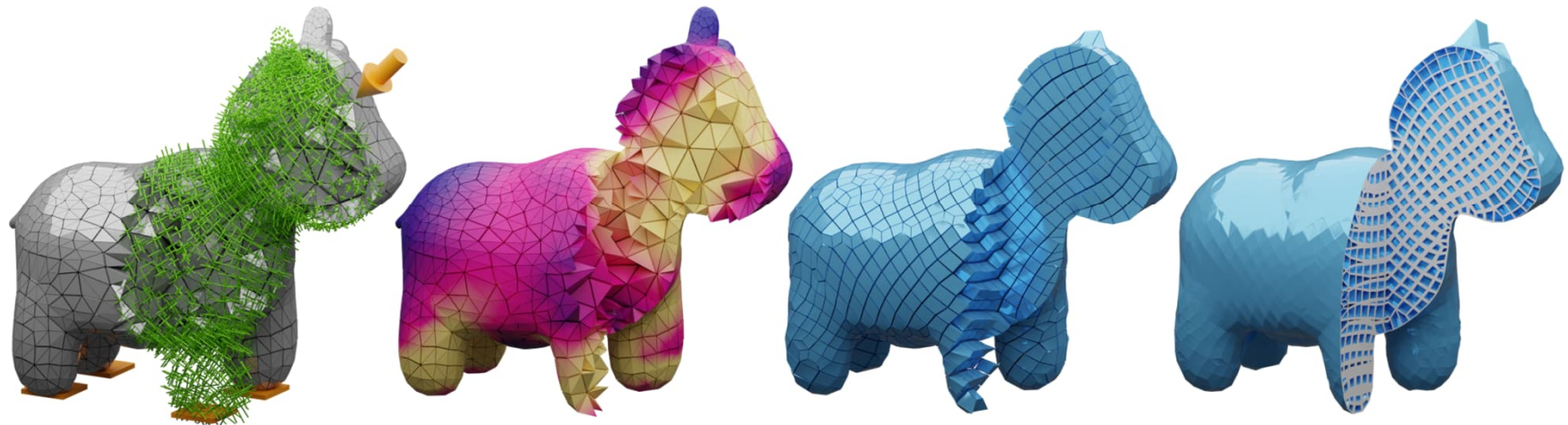


[Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints](#)

Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi



Slide for Future: Sustainability in CG (@ EG25)



[Stress-Aligned Hexahedral Lattice Structures](#)

Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann